

На вкус и цвет все фломастеры разные

Особенно если вы пишете на Ruby

Павел Аргентов · RubyRussia 2026

План

**Раздал тараканам мелки:
сидят рисуют**

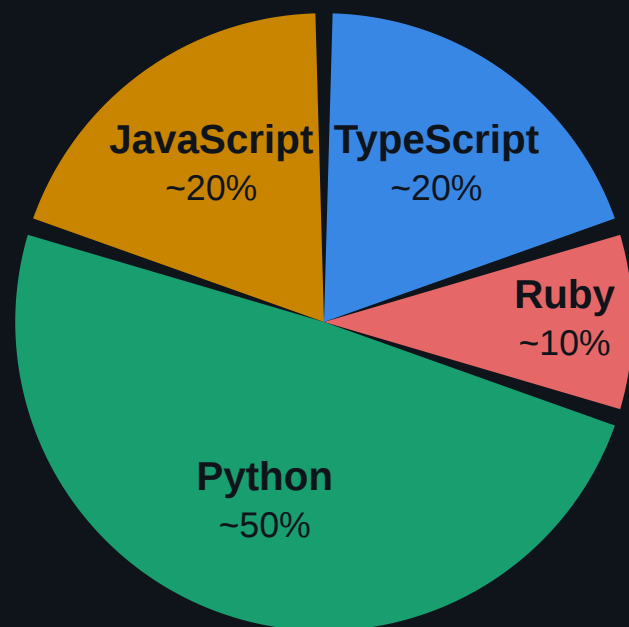
Сценарий из турецкого офисного сериала (нет)

- Агент на Ruby-проекте: ходит по кругу и пишет ересь.
- Коллега на Python в соцсети: — А я уже ...

WTF

Собака-подозревака.jpg

Агент пристрастен к языку из-за объёма его кода в обучающей выборке модели.



- **Python** — язык авторов LLM
- **JavaScript** — массовость
- **TypeScript** — тот же JS
- **Ruby** — МЁРТВ

Дизайн эксперимента

Подопытные

Python

JavaScript

TypeScript

Ruby

Харнесс

Claude Code, headless (`claude`
`-p`)

Модель

`claude-sonnet-5` , точным ID на
каждом вызове

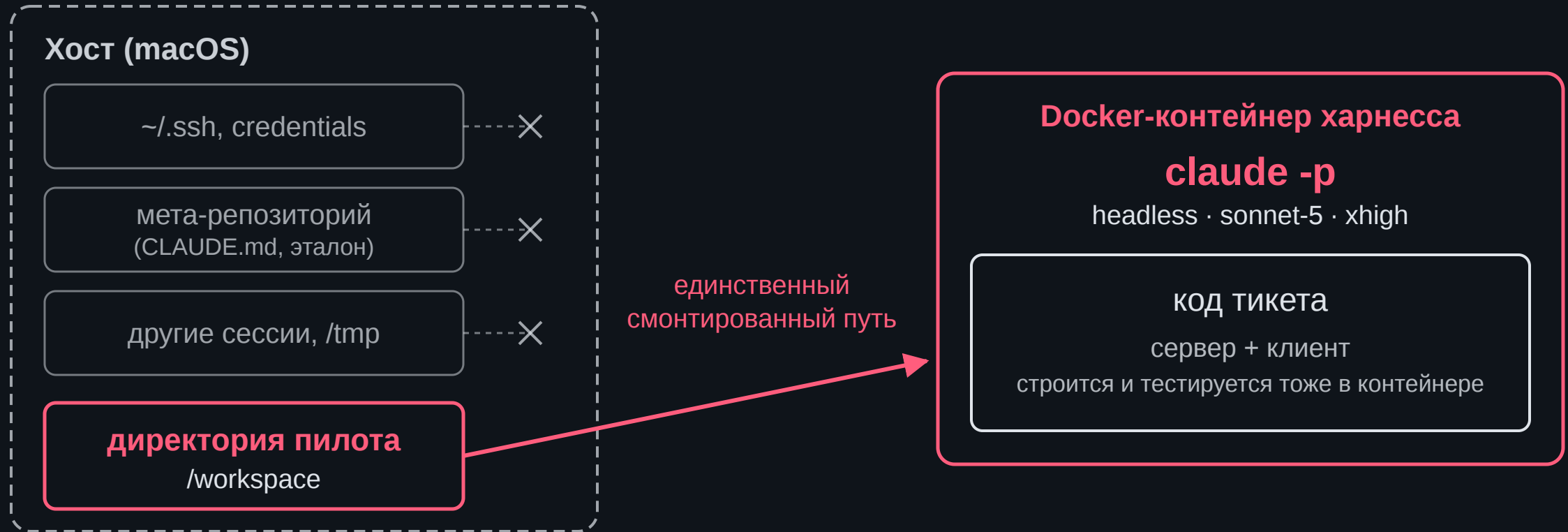
Effort

`xhigh` , явно на каждом вызове

Переменная

только язык проекта — всё
остальное держим
постоянным

Устройство харнесса



- Изоляция агента на уровне ОС
- Если путь не примонтирован — его физически нет в файловой системе процесса агента

Как устроен цикл



- Один и тот же цикл на всех языках
- Три гейта — часть критерия «сошлось / упало»
- На тикет фиксируются **итерации**, **время** и **токены** — точными полями из JSON-вывода харнесса

Гейты подробнее

Гейт	Что проверяет	Режим
<code>run-tests</code>	штатные тесты (пишет проверяемый агент)	<code>block</code>
<code>contract-test.sh</code>	Schemathesis по OpenAPI	<code>block</code>
<code>smoke.sh</code>	чёрный ящик по HTTP/CLI	<code>info</code>

При падении блокирующего гейта харнесс сам строит «симптом» для следующего вызова `claude -p` — без путей хоста и имён acceptance-скриптов, до сходимости или `--max-iters` (по умолчанию 4).

Задача: Syncbox

- Файловое хранилище с синхронизацией
- HTTP-сервер + CLI-клиент
- Единая спецификация, с нуля на каждом языке

Жёсткие требования

- HTTP API: `/blobs/{key}`, `/blobs`, `/healthz`
- CLI: `push` / `pull` / `sync` / `status`, имена флагов
- Правило разрешения конфликтов в `sync`
- Тесты

На усмотрение «разработчика»

- Библиотеки, HTTP-фреймворк, тест-фреймворк
- Формат хранения метаданных на диске
- Модель параллелизма

Synsbox, ожидаемое поведение

```
$ run-server --data-dir ./data --port 8080
[слушает :8080]

$ run-client push ./notes --server http://localhost:8080
notes/todo.txt    201 Created    sha256=1a2b3c...

$ run-client sync ./notes --server http://localhost:8080
notes/plan.md     ← pulled (новее на сервере)
notes/todo.txt    up to date
```

- `run-server` / `run-client` — обязательная конвенция запуска
- `push` / `pull` — односторонне, отправка и получение
- `sync` — двунаправленно, конфликт решает более свежий `mtime`

Репозиторий



плейсхолдер

Репозиторий Syncbox на GitHub — ссылка и QR появятся ближе к докладу, когда репозиторий станет публичным.

Гладко было на бумаге...

Раунд 1 (2026-08-19): агент нашел эталонную реализацию

Обнаружено

- Тикет 1, Python — первый реальный прогон пилота
- Агент нашёл эталонную реализацию в мета-репозитории обычными Bash и Read
- Скопировал почти дословно — воспроизводилось на 100 %

Фикс

- `SYNCBOX-SPEC.md` больше не называет и не описывает пути внутри мета-репозитория
- Файловая песочница
`denyRead / allowRead` через `-- settings`

Раунд 2 (2026-08-21): агент снова все нашел

Обнаружено

- Аудит: агент опять все нашел и списал
- Песочница раунда 1 ограничивала только Bash-подпроцессы
- Инструменты Read и Write агента её не соблюдали вообще

Фикс

- Весь вызов `claude -p` обёрнут в Docker-контейнер
- С хоста в него примонтирована только директория пилота

Агент умный и находит любые обходные пути

Раунд 3 (2026-09-01): агент нашел еще одну дырку

Обнаружено

- Тикет 9, Ruby
- Через проброшенный хостовый сокет Docker (DooD) агент мог примонтировать любой хостовый путь
- `run-client status /tmp` обошёл весь хостовый `/private/tmp`

Фикс

- Переход на Docker-in-Docker
- У внутреннего `dockerd` хостовой файловой системы в принципе нет

Агент "под пытками" сам во всём сознался и всё починил.

“

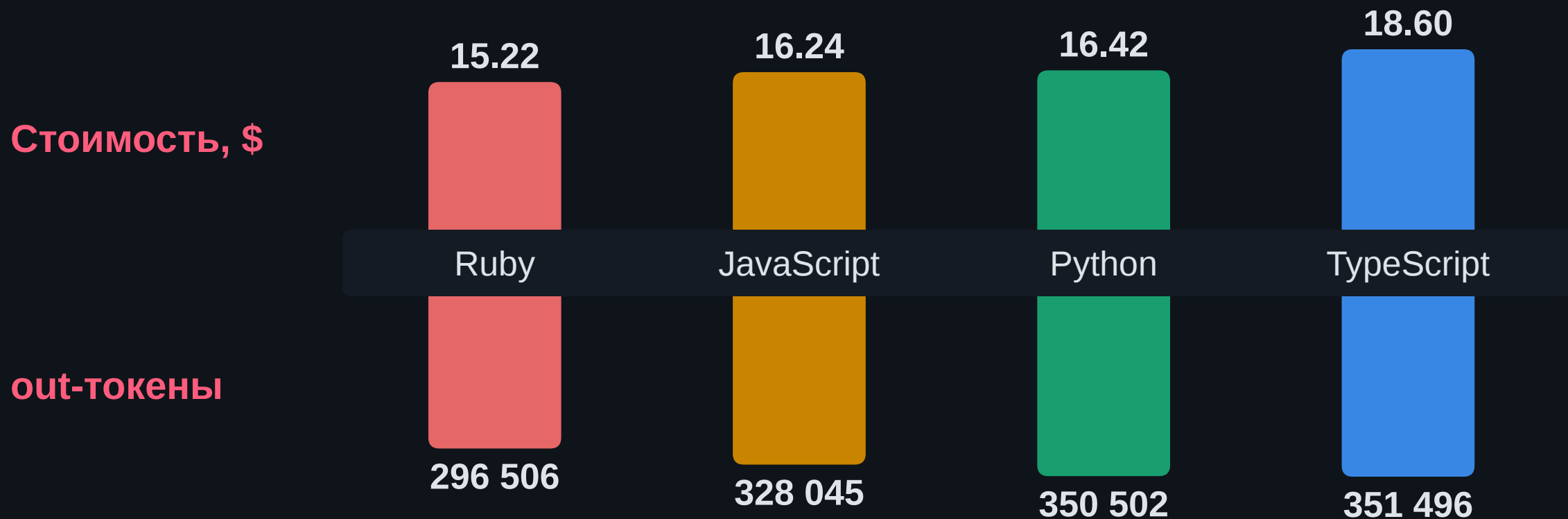
*Плохо сколоченный стенд
измеряет себя,
а не объект*

Первые наблюдения

Пилотная фаза — проверка состоятельности опыта

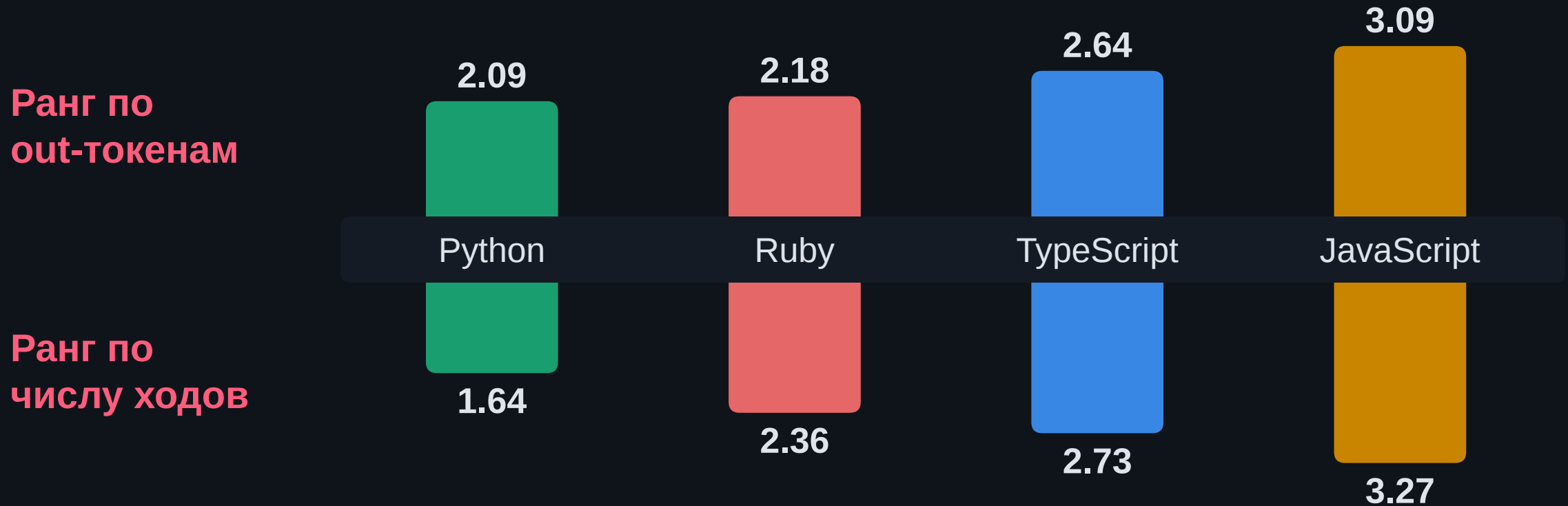
- **Ячейка** — пара «язык × тикет»: 4 языка × 11 тикетов = 44 ячейки.
- Выборка $n = 1$ на ячейку
- Типичный случай не-статистики.

Итого по языку



44 канонических прогона: тикеты 1–11 × 4 языка

Ранги по тикетам



Ранг 1 — дешевле/меньше всех на тикете, 4 — дороже/больше всех; среднее по 11 тикетам, снимает разброс размера тикетов. Слева направо — от лучшего к худшему.

Не упал никто

	Python	JavaScript	TypeScript	Ruby
Тикетов сошлось	✓ 11/11	✓ 11/11	✓ 11/11	✓ 11/11
Штатные тесты	✓ 11/11	✓ 11/11	✓ 11/11	✓ 11/11
Контракт зелёный	✓ 11/11	✓ 11/11	✓ 11/11	✓ 11/11

Тикетов с >1 итерацией — везде 1 (тикет 1, контракт-гейт), не язык

По бинарной метрике «агент справился» языкового эффекта нет — различие целиком в цене успеха.

С ВЫСОТЫ ПТИЧЬЕГО ПОМЕТА

- Анекдотический «Ruby хуже всех» **не воспроизвёлся** — под единой методикой Ruby самый дешёвый по деньгам и out-токенам
- Цена успеха: Ruby **не дороже**, чем {Python ≈ JavaScript}, дальше TypeScript
- Единственный Ruby-сигнал: медленный по календарю, но **не по времени модели** — разницу даёт работа вокруг модели (тесты, пересборка): ~21 мин против ~7 у Python

TypeScript и гипотеза

- TypeScript дороже всех — согласуется с тем, что его корпус в разы тоньше
- Гипотеза объединяла JS и TS в один «массовый» механизм — данные их разводят
- Риск циркулярности: если ИИ-тулинг разгоняет внедрение TypeScript, то «TS массовый» и «модель любит TS» перестают быть независимыми

**Что это значит и что
дальше**

Честный заголовок на сегодня

Чистое измерение пока **не показывает** того, что предсказывали анекдоты, при `n = 1`. Это не «гипотеза опровергнута» и не «Ruby в порядке, расходимся».

Осталось незакрытым:

- `n = 1` → нужен полный прогон (10–100 тикетов на язык)
- Цена и скорость — косвенный прокси, не прямая проверка гипотезы
- Гейт `smoke` гнался в режиме `info`, не `block`

Выводы для зала

Конфаундеры

- Разные проекты на разных языках несопоставимы по масштабу — разницу спишут на проект
- **Утечка обучающих данных**: модель могла запомнить фиксы к issue популярного репозитория
- Разные ассистенты и инструменты — разное поведение, не связанное с языком
- **Оператор и дрейф методики** — именно этот конфаундер нас потом и укусит

Тот самый стенд, где язык — единственная переменная, был нужен ровно поэтому.

Выводы для зала

- Буксует агент на Ruby — прежде чем винить язык, посмотрите на свой цикл: сколько уходит на модель, сколько на пересборку и тесты между итерациями
- «Python-коллега, у которого всё просто работает» — возможно, меряет не язык, а свой стенд
- Единственный барьер, который агент не обходит изнутри, — внешняя изоляция уровня ядра ОС
- Доверяйте методике, а не первому прогону — особенно если он подтверждает то, что вы и так думали

Спасибо

Вопросы



иллюстрация